

Clase 119 — Buffer overflow en stack: teoría

Parte: 5 — Explotación de sistemas y binarios · Fuente: Erickson, Hacking 2e · Aleph One, "Smashing the Stack" 🕒 Duración estimada: 100 min · Nivel: Intermedio

Objetivo

Entender con precisión **por qué** y **cómo** un buffer overflow en el stack corrompe la dirección de retorno y permite secuestrar el flujo de ejecución. Esta clase es teórica: sienta el modelo mental (layout de memoria, escritura fuera de límites, control de `RIP`) que aplicarás prácticamente en la clase 120. Sin este mapa conceptual, la explotación es prueba y error a ciegas.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** cómo una escritura sin verificación de longitud sobrepasa un buffer local.
2. **Dibujar** el layout del stack y señalar qué bytes controlan `RIP`.
3. **Definir** el *offset* al retorno y por qué es el número clave del exploit.
4. **Clasificar** funciones peligrosas de C (`gets`, `strcpy`, `sprintf`, `scanf %s`).
5. **Relacionar** el overflow con las mitigaciones que lo dificultan (adelanto de la clase 122).

Temas

#	Tema	Por qué importa
1	Buffers en el stack	Dónde se guardan y su tamaño fijo
2	Escritura fuera de límites	La causa raíz del bug
3	Sobrescritura de saved RBP y ret	Camino hacia el control de RIP
4	Concepto de offset	Cuántos bytes hasta el retorno
5	Control de RIP → control de flujo	El objetivo del exploit
6	Funciones peligrosas de C	Dónde nacen estos bugs
7	Impacto y variantes	Local vs remoto, DoS vs RCE
8	Panorama de mitigaciones	Por qué hoy no basta lo básico

Definiciones y características

- **Buffer overflow:** escritura de más datos de los que caben en un buffer, invadiendo memoria adyacente. *Clave:* en el stack, esa memoria suele ser saved RBP y la dirección de retorno.
- **Out-of-bounds write:** acceso de escritura fuera del rango válido de un objeto. *Clave:* CWE-787, consistentemente entre las debilidades más peligrosas.

- **Offset al retorno:** distancia en bytes desde el inicio del buffer hasta la dirección de retorno. *Clave:* `tamaño_buffer + padding + saved RBP`.
- **Control de RIP:** lograr que la CPU ejecute una dirección elegida por el atacante. *Clave:* con RIP controlado se salta a shellcode, `win()`, o una cadena ROP.
- **Función insegura:** rutina que copia sin límite (`gets`, `strcpy`, `strcat`, `sprintf`). *Clave:* sus versiones acotadas (`fgets`, `strncpy`, `snprintf`) mitigan el problema.
- **NOP sled:** relleno de instrucciones `nop` que amplía el margen de acierto al saltar a shellcode. *Clave:* útil cuando la dirección exacta es incierta.

Herramientas y preparación

Esta clase es conceptual, pero conviene tener listos GDB+pwndbg (clase 118) y un editor. Prepara un diagrama en papel o en un `.md` del stack para razonar los offsets.

```
# Repasa las protecciones que trae un binario cualquiera:
checksec --file=/bin/ls    # instalar: pip install pwntools (trae checksec)
```

Laboratorio guiado

Ejercicio conceptual-aplicado (sin lanzar exploit todavía).

1. Toma el binario `vuln` de la clase 118 y ábrelo en pwndbg.
2. Desensambla `vuln` y anota el tamaño reservado para el buffer (`sub rsp, 0x50` → 80 bytes).
3. Dibuja en papel el frame de `vuln` de arriba (direcciones altas) a abajo:

```
text [ ret address ] ← rbp+8 (objetivo) [ saved RBP ] ← rbp [ buf[63..0] ] ← rbp-0x40 ...
crece hacia rbp
```

4. Calcula el offset teórico al retorno y contrástalo con el valor que hallaste con `cyclic` en la 118.
5. Identifica en el código fuente qué función provoca el overflow (`gets`) y qué la haría segura (`fgets`).
6. Ejecuta `checksec ./vuln` y anota qué mitigaciones están **desactivadas** (NX, canary, PIE) y por qué eso hace el binario explotable — lo aprovecharás en la clase 120.
7. Escribe un breve informe (5-8 líneas) explicando la cadena causa→efecto del bug.

Ejercicios

1. Explica por qué `strncpy(dst, src, sizeof(dst))` no siempre es seguro (terminación nula).
2. Dado un buffer de 32 bytes y saved RBP de 8, ¿cuál es el offset al retorno en x64?
3. Lista cinco funciones de C inseguras y su reemplazo acotado.
4. Describe la diferencia entre un overflow que causa DoS y uno que logra RCE.
5. ¿Por qué un NOP sled aumenta la fiabilidad? ¿Cuándo no ayuda?
6. Relaciona cada mitigación (NX, canary, ASLR, PIE) con qué parte del ataque bloquea.

Reto verificable

Redacta y entrega un diagrama del stack frame de `vuln` con offsets numéricos exactos y una frase que indique cuántos bytes hay que escribir para alcanzar (sin sobrescribir aún) la dirección de retorno.

Criterio de aceptación: el offset del diagrama coincide con el que confirma `cyclic -l` en GDB, y señalas correctamente la posición de saved RBP y ret address.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Offset calculado $\neq$ real	Olvidaste el saved RBP (8 bytes) o el padding de alineación
"Mi overflow no controla RIP"	Hay stack canary; lo verás en checksec (clase 122)
Confundir dirección alta/baja	El stack crece hacia abajo; dibújalo siempre
Creer que <code>strcpy</code> es infalible	Puede dejar la cadena sin <code>\0</code> ; revisa longitudes
Pensar que todo overflow = RCE	Muchos solo causan crash/DoS; depende del control logrado

Preguntas frecuentes

 **¿Por qué sobrescribir el retorno y no otra cosa?** Porque `ret` carga esa dirección en `RIP` directamente, dándote control de flujo sin trucos adicionales.

 **¿Esto sigue funcionando en binarios modernos?** No tal cual: canarios, NX, ASLR y PIE lo complican. Por eso primero se estudia en un binario sin protecciones.

 **¿Es lo mismo overflow en stack que en heap?** No; el heap tiene metadatos y mecánica distinta (clases 126-127).

Referencias

- Aleph One, "Smashing the Stack for Fun and Profit", *Phrack* 49 — <http://phrack.org/issues/49/14.html>
- Erickson, J. *Hacking: The Art of Exploitation*, 2e, cap. 0x3. No Starch Press.
- CWE-787: Out-of-bounds Write — <https://cwe.mitre.org/data/definitions/787.html>
- OWASP, Buffer Overflow — https://owasp.org/www-community/vulnerabilities/Buffer_Overflow

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Siguiente clase

[Clase 120 - Buffer overflow en stack: explotacion practica](#)